PROJE PLANLAMA DOKÜMANI · V1.0

Akıllı Depo Pick-to-Light Sistemi

Işığa duyarlı ürün toplama ve anlık stok yönetimi
Sipariş numarasına göre LED yönlendirme · otomatik stok düşme · kırmızı alarm

C# · .NET 9 | ASP.NET Core Web API | Next.js 15 + React 19 | PostgreSQL 17 | MQTT | ESP32 | SignalR

DOKÜMAN
Teknik Planlama & Yol Haritası

KAPSAM
Yazılım + Elektrik ortak projesi

SÜRE
8 hafta · prototip teslimi

DURUM
Planlama (kodlama başlamadı)

İçindekiler

1 Yönetici Özeti ve Proje Hedefi

2 Kapsam, Varsayımlar ve Kapsam Dışı

3 Gereksinimler (Fonksiyonel / Fonksiyonel Olmayan)

4 Sistem Mimarisi ve Teknoloji Yığını

5 Veri Modeli ve PostgreSQL Şeması

6 Çekirdek İş Mantığı: Işık Kararı ve Atomik Stok Düşme

7 Sipariş Yaşam Döngüsü ve Durum Makineleri

8 Backend Tasarımı (C# / .NET) ve REST API Sözleşmesi

9 MQTT Haberleşme Sözleşmesi

10 Donanım Planı ve ESP32 Firmware Tasarımı

11 Frontend Tasarımı (Next.js) ve Ekran Planı

12 Güvenlik, Yetkilendirme ve İzlenebilirlik

13 Hata Senaryoları ve Kurtarma Stratejisi

14 Test Planı ve Kabul Kriterleri

15 8 Haftalık Proje Planı ve İş Kırılım Yapısı

16 Risk Analizi

17 Geliştirme Ortamı, Repo Yapısı ve Çalışma Düzeni

18 Demo Senaryosu ve Başarı Ölçütleri

19 Sözlük

Teknoloji yığını hakkında önemli not

Hafta-1 sunumunda backend **Spring Boot** olarak gösterilmişti. Bu planlama dokümanı, güncel talebiniz doğrultusunda **C# / .NET 9 + Next.js + PostgreSQL** yığını üzerine kurulmuştur: **backend Java/Spring Boot yerine C# /.NET** ile yazılacak, **arayüz Next.js olarak korunmuştur**. Veri modeli (ER diyagramı), ışık mantığı, MQTT akışı ve donanım tarafı değişmemiştir. Sunumun 7. ve 9. slaytlarındaki backend etiketi güncellenmiştir (bkz. güncel sunum PDF'i).

1. Yönetici Özeti ve Proje Hedefi

Operatörün ekran taramak yerine yanan ışığı takip ettiği, stoğu kendi düşen bir depo toplama sistemi.

1.1 Problem

E-ticaret depolarında sipariş hazırlama (order picking) süresinin büyük bölümü **ürünü aramakla** geçer. Elle yapılan toplamada üç temel sorun vardır: raf arama kaynaklı zaman kaybı, yanlış ürün/yanlış adet kaynaklı insan hatası ve stoğun anlık görülememesinden doğan "kör stok" durumu — eksik, ancak sipariş anında fark edilir.

1.2 Çözüm

Her envanter kutusunun başına bir durum LED'i ve bir onay sensörü yerleştirilir. Sisteme sipariş numarası girildiğinde yazılım, siparişteki her kalem için ilgili kutuyu bulur, stok durumunu değerlendirir ve kutunun LED'ini üç renkten biriyle yakar:

YEŞİL **Stok yeterli**
Ürün rahatça verilir, sistem stoğu otomatik düşer.

SARI **Stok azaldı**
Sipariş karşılanır ama kritik seviye; uyarı ve tedarik talebi üretilir.

KIRMIZI **Stok yetersiz**
Alarm; kalem kilitlenir, ekrana uyarı basılır, tedarik tetiklenir.

1.3 Proje hedefi (8. hafta sonunda ulaşılacak nokta)

> Sipariş numarasını gören, doğru kutuyu ışıkla gösteren, operatörün onayıyla stoğu **kendi düşen**, yetersiz stokta **alarm veren** ve tüm hareketi loglayan, en az **4 kutuluk** çalışan bir fiziksel prototip + web arayüzü.

1.4 Ölçülebilir hedefler

< 1 sn	**0**	**%100**	**≥ 4**
sipariş girişinden LED yanmasına kadar geçen süre	negatif stok / fazla verme vakası (eşzamanlı işlemlerde)	stok hareketinin loglanma oranı (izlenebilirlik)	eşzamanlı çalışan fiziksel kutu sayısı

1.5 Ekipler ve sorumluluk sınırı

Ekip	Sorumluluk	Teslim ettiği arayüz
Elektrik Mühendisliği	Kare modüler kutu tasarımı ve montajı, LED sürücü devresi, ESP32 programlama, onay sensörü (buton / IR / ağırlık) entegrasyonu, 5 V güç dağıtımı	MQTT üzerinden konuşan, komutla LED yakan ve teyit mesajı üreten kutu
Yazılım Mühendisliği	Backend API ve iş mantığı, veritabanı şeması, sipariş & otomatik stok düşme akışı, web arayüzü, alarm ve bildirim servisi	MQTT komut sözleşmesi + web arayüzü + stok veritabanı

İki ekibin buluştuğu tek nokta MQTT mesaj sözleşmesidir (Bölüm 9). Bu sözleşme 2. hafta sonunda dondurulur; böylece iki ekip birbirini beklemeden paralel ilerleyebilir.

2. Kapsam, Varsayımlar ve Kapsam Dışı

Ne yapacağız, neyi bilerek yapmıyoruz.

2.1 Kapsam içi

- Sipariş girişi (manuel sipariş numarası + kalemleri) ve sipariş listeleme
- Ürün, kutu ve stok envanterinin yönetimi (CRUD + sayım)
- Sipariş numarasına göre kutu eşleştirme ve LED yakma
- Üç renkli stok durumu kararı (yeşil / sarı / kırmızı) ve eşik yönetimi
- Operatör teyidi sonrası otomatik ve atomik stok düşme
- Yetersiz stokta ekran alarmı + tedarik talebi kaydı
- Tüm stok hareketlerinin ve cihaz olaylarının loglanması
- Canlı web arayüzü (kutu durumları, aktif sipariş, alarm paneli)
- ESP32 tabanlı fiziksel kutu prototipi (en az 4 adet)

2.2 Kapsam dışı (bu 8 haftada yapılmayacak)

- Gerçek e-ticaret platformu entegrasyonu (Trendyol, Shopify vb. API'leri) — sipariş **manuel** veya seed verisiyle girilir
- Kargo/sevkiyat, fatura, muhasebe ve iade süreçleri
- Barkod/RFID ile ürün doğrulama (NFC alanı şemada hazır bırakıldı, kullanılmayacak)
- Çok depolu / çok lokasyonlu yapı — tek depo varsayılır
- Rota optimizasyonu (en kısa toplama yolu hesaplama)
- Mobil uygulama; arayüz responsive web olarak kalır
- Bulut dağıtımı ve yüksek erişilebilirlik; sistem yerel ağda (LAN) çalışır

2.3 Varsayımlar

Varsayım	Gerekçe / etkisi
Bir kutuda tek çeşit ürün bulunur	ER'de `kutu.urun_id` tekil FK. Çoklu ürün gerekirse ara tablo açılması gerekir — v2 konusu.
Bir ürün birden çok kutuda bulunabilir	`urun 1–N kutu`. Toplama sırasında stoğu en uygun kutu seçilir (Bölüm 6.4).
Depo yerel ağı (Wi-Fi) kesintisiz ve ESP32 menzilindedir	Kopma durumunda LWT ile kutu OFFLINE işaretlenir, sipariş o kutu için elle yürütülür.
Operatör sayısı azdır ve sisteme giriş yapar	Her stok hareketi bir personele bağlanır (izlenebilirlik).
Aynı anda birden fazla sipariş toplanabilir	Bu yüzden stok düşme **atomik** olmak zorundadır (Bölüm 6.2).
Güvenlik stoğu (buffer) ürün bazında tanımlanır	`urun.guvenlik_stogu`, varsayılan 5. Sarı ışık eşiğini bu belirler.

2.4 Paydaşlar

Paydaş	Beklentisi	Sistemdeki rolü
Depo operatörü	Ekran okumadan, ışığı takip ederek hızlı toplama	`OPERATOR` — sipariş toplar, teyit verir
Depo sorumlusu	Anlık stok görünürlüğü, eksik uyarısı, tedarik takibi	`ADMIN` — ürün/kutu/eşik yönetimi, raporlar
Proje danışmanı / jüri	Çalışan prototip, teknik derinlik, iki disiplinin entegrasyonu	Demo ve değerlendirme

3. Gereksinimler

Her gereksinim numaralıdır; test planındaki kabul kriterleri bu numaralara referans verir.

3.1 Fonksiyonel gereksinimler

No	Gereksinim	Kabul ölçütü
FR-01	Sisteme sipariş numarası ve kalemleriyle sipariş girilebilir	Aynı `siparis_no` ikinci kez girilemez (UK ihlali kullanıcıya anlaşılır hata döner)
FR-02	Sipariş başlatıldığında her kalem bir kutuyla eşleştirilir	Kalem kaydında `kutu_id` dolu; uygun kutu yoksa kalem `YETERSIZ` işaretlenir
FR-03	Stok durumu üç renkten biriyle belirlenir	s≥q+b → YEŞİL, q≤s<q+b → SARI, s<q → KIRMIZI (Bölüm 6.1)
FR-04	Kutunun LED'i hesaplanan renkte yakılır	Komut MQTT ile yayınlanır; cihaz 1 sn içinde ışığı yakar
FR-05	Yeşil/sarı durumda ürün alındığında stok otomatik düşer	Teyit sonrası `stok.adet` azalır ve `stok_hareketi` kaydı oluşur
FR-06	Stok düşme işlemi atomiktir ve negatife düşmez	100 eşzamanlı istekte toplam düşülen miktar mevcut stoğu aşmaz
FR-07	Stok yetersizse ekrana uyarı döner ve alarm kaydı oluşur	`alarm` tablosuna `STOK_YOK`, arayüzde anlık kırmızı bildirim
FR-08	Stok güvenlik seviyesinin altına düştüğünde tedarik talebi açılır	`tedarik_talebi` kaydı `ACIK` durumda; aynı ürün için mükerrer açılmaz
FR-09	Operatör teyidi buton / IR / ağırlık sensöründen gelebilir	`teyit_yontemi` alanı doğru kaydedilir; arayüzden manuel teyit de mümkündür
FR-10	Arayüzde kutu durumları ve stok canlı izlenir	Sayfa yenilemeden güncellenir (SignalR), gecikme < 1 sn
FR-11	Tüm stok hareketleri (çıkış/giriş/sayım) loglanır	Kayıtta önceki ve sonraki adet, personel, zaman bulunur
FR-12	Cihazların bağlantı durumu izlenir	Kutu ONLINE/OFFLINE gösterilir; LWT ile 30 sn içinde OFFLINE'a düşer
FR-13	Sipariş iptal edilebilir ve rezerve stok iade edilir	İptal sonrası stok eski değerine döner, `GIRIS` hareketi loglanır
FR-14	Stok sayımı (düzeltme) yapılabilir	`SAYIM` tipinde hareket; fark loglanır
FR-15	Kullanıcı rolüne göre yetkilendirme yapılır	OPERATOR stok eşiği/ürün değiştiremez; ADMIN tüm işlemleri yapar
FR-16	Kutu LED'i test edilebilir (bakım modu)	Arayüzden tek kutuya test komutu gönderilir

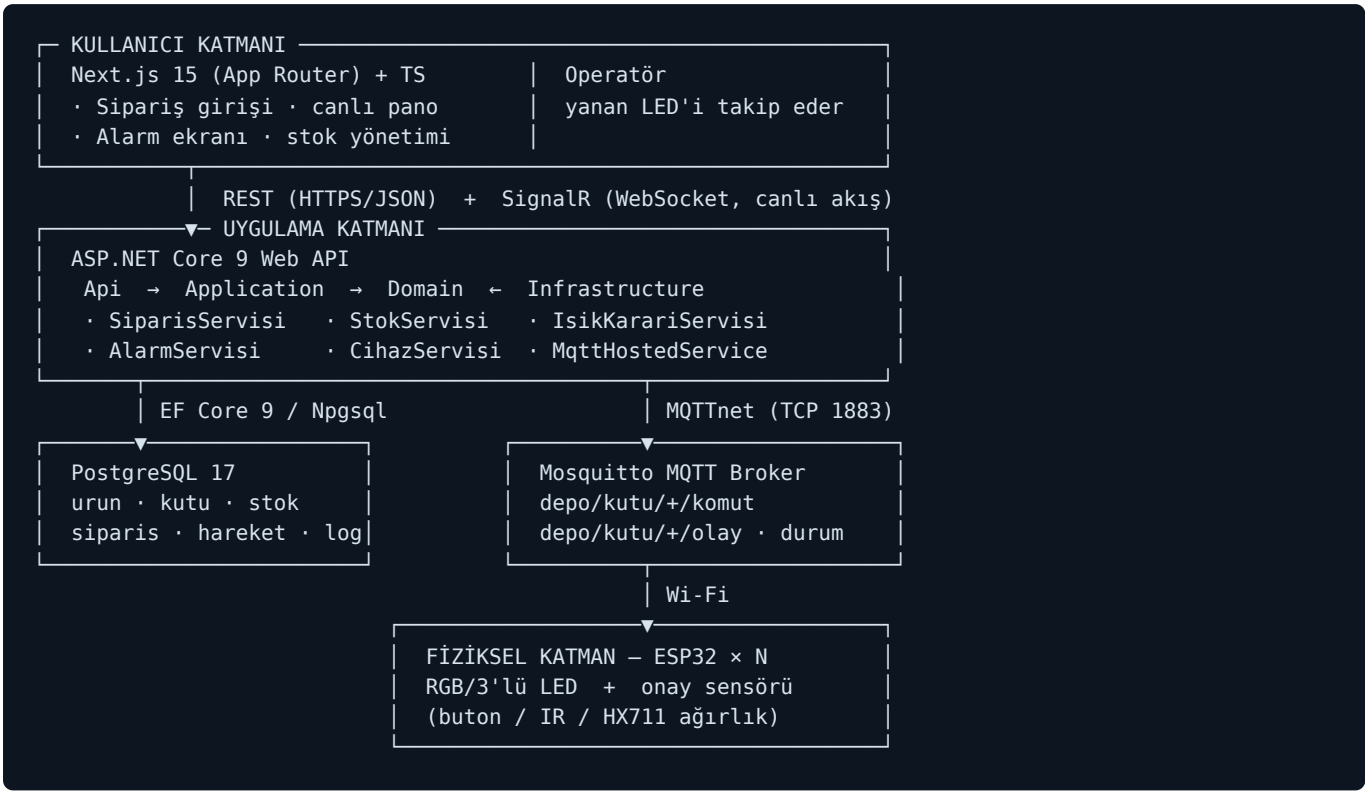
3.2 Fonksiyonel olmayan gereksinimler

No	Başlık	Hedef
NFR-01	Performans	Sipariş başlatmadan LED yanmasına uçtan uca < 1 sn (yerel ağ); API p95 < 200 ms
NFR-02	Tutarlılık	Stok işlemleri ACID; eşzamanlılıkta negatif stok imkânsız (DB seviyesinde `CHECK adet >= 0`)
NFR-03	İdempotenlik	Aynı `mesaj_id` ile gelen tekrar teyit mesajı stoğu ikinci kez düşürmez
NFR-04	Dayanıklılık	MQTT broker veya cihaz koptuğunda sistem veri kaybetmez; komutlar QoS 1 ile tekrar denenir
NFR-05	İzlenebilirlik	Her stok değişimi kim/ne zaman/hangi sipariş bilgisiyle geri izlenebilir
NFR-06	Ölçeklenebilirlik	Kutu eklemek yalnızca veritabanına kayıt + cihaz kurulumu gerektirir; kod değişmez
NFR-07	Güvenlik	JWT kimlik doğrulama, rol bazlı yetki, cihaz başına MQTT kimlik bilgisi
NFR-08	Kullanılabilirlik	Operatör ekranı 3 m mesafeden okunabilir; renk körlüğü için renk + ikon + metin birlikte
NFR-09	Kurulabilirlik	`docker compose up` ile tüm sistem (DB + broker + API) tek komutta ayağa kalkar
NFR-10	Test edilebilirlik	Donanım olmadan sanal kutu simülatörü ile uçtan uca test yapılabilir

4. Sistem Mimarisi ve Teknoloji Yığını

Üç katman: kullanıcı, uygulama, fiziksel. Katmanlar arası tek temas noktaları REST/SignalR ve MQTT'dir.

4.1 Katman şeması



4.2 Teknoloji seçimleri ve gerekçeleri

Katman	Seçim	Gerekçe
Backend dili/çatısı	C# 13 · .NET 9 · ASP.NET Core Web API	Güçlü tipleme, olgun DI ve hosted service altyapısı (MQTT dinleyicisi için ideal), yüksek performans
ORM	EF Core 9 + Npgsql	Migration ile şema versiyonlama; ham SQL'e de izin verir (atomik stok düşme için gerekli)
Canlı iletişim	SignalR	.NET ile yerel entegre WebSocket; arayüze anlık stok/alarm/LED durumu push edilir
MQTT istemcisi	MQTTnet 5 (`IHostedService` olarak)	.NET'in standart MQTT kütüphanesi; QoS 1, LWT ve yeniden bağlanma desteği
Veritabanı	PostgreSQL 17	ACID, `UPDATE ... RETURNING`, kısıt (CHECK) desteği, JSONB ile ham MQTT payload saklama
Doğrulama / log	FluentValidation · Serilog	İstek doğrulama ve yapılandırılmış log (dosya + konsol)
Frontend	Next.js 15 (App Router) + React 19 + TypeScript	Dosya tabanlı yönlendirme, sunucu/istemci bileşen ayrımı, tek komutla üretim derlemesi; tip güvenliği API DTO'larıyla paylaşılır
Veri/stat yönetimi	TanStack Query + @microsoft/signalr	Sunucu durumu önbellekleme + canlı olayla invalidate etme kalıbı
Stil	Tailwind CSS	Kutu ızgarası ve renk durumlarını hızlı kurmak için
Broker	Eclipse Mosquitto 2	Hafif, ESP32 ile sorunsuz, Docker imajı hazır
Firmware	ESP32 · Arduino framework (PlatformIO)	Elektrik ekibinin bilgi birikimi; `PubSubClient` / `Adafruit_NeoPixel` kütüphaneleri
Test	xUnit · FluentAssertions · Testcontainers · Playwright	Birim + gerçek PostgreSQL/Mosquitto ile entegrasyon + E2E
Ortam	Docker Compose	DB + broker + API + frontend tek komutla

4.3 Backend proje yapısı (Clean Architecture, 4 katman)

```
PickToLight.sln
├─ src/PickToLight.Domain          // Entity, enum, iş kuralları — bağımlılığı yok
│   ├─ Entities/ (Urun, Kutu, Stok, Siparis, SiparisKalemi, StokHareketi, Alarm...)
│   ├─ Enums/    (LedRengi, SiparisDurumu, KalemDurumu, HareketTipi, AlarmTipi)
│   └─ Kurallar/ (IsikKarari.cs  → saf fonksiyon, test edilmesi kolay)
├─ src/PickToLight.Application     // Use-case'ler, DTO, arayüzler
│   ├─ Siparisler/ (SiparisOlustur, SiparisBaslat, KalemTeyitEt, SiparisIptal)
│   ├─ Stoklar/    (StokDus, StokIade, StokSayim)
│   ├─ Abstractions/ (IMqttYayinci, IBildirimYayinci, IStokDeposu)
│   └─ Dtos/ + Validators/
├─ src/PickToLight.Infrastructure  // Dış dünya
│   ├─ Persistence/ (AppDbContext, Configurations, Migrations, StokDeposu)
│   ├─ Mqtt/        (MqttHostedService, MqttYayinci, OlayIsleyici)
│   └─ Realtime/    (SignalRBildirimYayinci)
└─ src/PickToLight.Api             // Giriş noktası
    ├─ Controllers/ · Hubs/DepoHub.cs · Middleware/HataYakalama.cs
    └─ Program.cs (DI, auth, CORS, Swagger)
tests/  Domain.UnitTests · Application.UnitTests · Api.IntegrationTests
```

Bağımlılık yönü daima içe doğrudur: Api → Application → Domain. Infrastructure yalnızca Application'daki arayüzleri uygular. Böylece ışık kararı ve stok kuralları veritabanı/MQTT olmadan test edilebilir.

5. Veri Modeli ve PostgreSQL Şeması

Hafta-2 ER diyagramı esas alınmıştır; aşağıda tablo tablo alan sözlüğü, kısıtlar, indeksler ve önerilen küçük eklemeler yer alır.

5.1 Tablolar ve rolleri

Tablo	Grup	Amacı
`urun`	Envanter	Ürün kataloğu; SKU, ad, birim ağırlık ve **güvenlik stoğu (b)** burada tutulur
`kutu`	Envanter	Fiziksel kare kutu; ESP32 kimliği (MAC), MQTT topic'i, raf konumu, bağlantı durumu
`stok`	Envanter	Kutu başına **mevcut adet (s)**; kutu ile 1:1. Optimistic lock için `versiyon` alanı
`siparis`	Sipariş akışı	Sipariş başlığı; benzersiz sipariş numarası ve durum
`siparis_kalemi`	Sipariş akışı	Hangi üründen kaç adet (**q**), hangi kutudan, hangi LED rengiyle, hangi yöntemle teyit edildi
`personel`	Kullanıcı	Operatör/yönetici; her hareket bir personele bağlanır
`stok_hareketi`	Log	Değişmez (append-only) hareket defteri: önceki adet, sonraki adet, tip, zaman
`alarm`	Log	STOK_AZALDI / STOK_YOK / CIHAZ_OFFLINE / TEYIT_ZAMAN_ASIMI bildirimleri
`tedarik_talebi`	Log	Eksik ürün için açılan ikmal talebi; ACIK → VERILDI → KAPANDI
`cihaz_olayi`	Log	Ham MQTT payload'ı (JSONB); hata ayıklama ve idempotenlik kontrolü için

5.2 Kritik alanlar ve kısıtlar

Alan	Tip	Kısıt / kural
`urun.sku`	varchar(50)	UNIQUE, NOT NULL
`urun.guvenlik_stogu` (b)	int	DEFAULT 5, CHECK ≥ 0 — sarı ışık eşiğini belirler
`kutu.cihaz_mac`	varchar(17)	UNIQUE — cihaz kimliği; MQTT mesajını kutuya bağlar
`kutu.mqtt_topic`	varchar(60)	UNIQUE, örn. `depo/kutu/1`
`kutu.baglanti_durumu`	enum	ONLINE | OFFLINE — LWT ile güncellenir
`stok.adet` (s)	int	**CHECK (adet >= 0)** — negatif stoğa karşı son savunma hattı
`stok.versiyon`	int	Her güncellemede +1 (optimistic concurrency)
`siparis.siparis_no`	varchar(30)	UNIQUE, NOT NULL
`siparis.durum`	enum	BEKLIYOR | HAZIRLANIYOR | TAMAMLANDI | KISMI_TAMAMLANDI | KILITLI | IPTAL
`siparis_kalemi.miktar` (q)	int	CHECK (miktar > 0)
`siparis_kalemi.led_rengi`	enum	YESIL | SARI | KIRMIZI
`siparis_kalemi.durum`	enum	BEKLIYOR | ISIK_YANDI | ALINDI | YETERSIZ | IPTAL
`stok_hareketi.tip`	enum	CIKIS | GIRIS | SAYIM
`cihaz_olayi.payload`	jsonb	Ham MQTT mesajı; GIN indeksi opsiyonel

5.3 ER diyagramına önerilen küçük eklemeler

Mevcut diyagram iş akışını karşılıyor. Uygulama sırasında şu beş ekleme hayatı kolaylaştırır:

Ekleme	Neden
`siparis_kalemi.sira` (int)	Operatörün kutuları hangi sırayla gezeceğini belirler; ileride rota optimizasyonuna açık kapı
`siparis_kalemi.mesaj_id` (uuid, UNIQUE)	MQTT teyit mesajının idempotenlik anahtarı — aynı teyit iki kez stok düşüremez
`stok.rezerve_adet` (int, v2)	İleride "rezerve ama henüz alınmamış" miktarı ayrı göstermek için. v1'de rezervasyon doğrudan `adet`'ten düşülecek (Bölüm 6.3)
`kutu.kapasite` (int)	Sayım ve doluluk göstergesi; ağırlık sensörüyle çapraz kontrol
`tedarik_talebi` kısmi UNIQUE	`UNIQUE (urun_id) WHERE durum = 'ACIK'` — aynı ürün için mükerrer açık talebi engeller

5.4 Örnek DDL (çekirdek tablolar)

```
CREATE TYPE led_rengi   AS ENUM ('YESIL','SARI','KIRMIZI');
CREATE TYPE hareket_tipi AS ENUM ('CIKIS','GIRIS','SAYIM');

CREATE TABLE urun (
  id              bigserial PRIMARY KEY,
  sku             varchar(50)  NOT NULL UNIQUE,
  ad              varchar(150) NOT NULL,
  birim_agirlik_g numeric(10,2),
  guvenlik_stogu  int NOT NULL DEFAULT 5 CHECK (guvenlik_stogu >= 0),
  olusturma       timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE kutu (
  id               bigserial PRIMARY KEY,
  kod              varchar(20) NOT NULL UNIQUE,
  urun_id          bigint REFERENCES urun(id),
  cihaz_mac        varchar(17) NOT NULL UNIQUE,
  mqtt_topic       varchar(60) NOT NULL UNIQUE,
  raf_konumu       varchar(30),
  baglanti_durumu  varchar(10) NOT NULL DEFAULT 'OFFLINE',
  son_gorulme      timestamptz
);

CREATE TABLE stok (
  kutu_id       bigint PRIMARY KEY REFERENCES kutu(id) ON DELETE CASCADE,
  adet          int NOT NULL CHECK (adet >= 0),   -- negatif stok imkânsız
  son_agirlik_g numeric(10,2),
  versiyon      int NOT NULL DEFAULT 0,
  guncelleme    timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE siparis_kalemi (
  id            bigserial PRIMARY KEY,
  siparis_id    bigint NOT NULL REFERENCES siparis(id) ON DELETE CASCADE,
  urun_id       bigint NOT NULL REFERENCES urun(id),
  kutu_id       bigint REFERENCES kutu(id),
  miktar        int NOT NULL CHECK (miktar > 0),
  led_rengi     led_rengi,
  durum         varchar(12) NOT NULL DEFAULT 'BEKLIYOR',
  teyit_yontemi varchar(10),
  mesaj_id      uuid UNIQUE,                      -- idempotenlik
  sira          int,
  alinma_zamani timestamptz
);
```

5.5 İndeks planı

İndeks	Hangi sorgu için
`idx_kalem_siparis ON siparis_kalemi(siparis_id)`	Sipariş detayının kalemlerini çekmek
`idx_kalem_durum ON siparis_kalemi(durum) WHERE durum='BEKLIYOR'`	Açık toplama görevlerini listelemek (kısmi indeks)
`idx_kutu_urun ON kutu(urun_id)`	Ürün için uygun kutu seçimi (Bölüm 6.4)
`idx_hareket_zaman ON stok_hareketi(zaman DESC)`	Hareket defteri ekranı, raporlar
`idx_hareket_kutu ON stok_hareketi(kutu_id, zaman DESC)`	Tek kutunun geçmişi
`idx_alarm_okunmamis ON alarm(okundu) WHERE okundu = false`	Alarm rozetindeki sayaç
`idx_siparis_no ON siparis(siparis_no)` (UK ile gelir)	Sipariş numarasıyla arama — ana giriş noktası

6. Çekirdek İş Mantığı: Işık Kararı ve Atomik Stok Düşme

Projenin kalbi burasıdır. Yanlış kurgulanırsa "ışık yeşil ama kutuda ürün yok" durumu doğar.

6.1 Işık kararı

`s` = kutudaki mevcut stok · `q` = sipariş kalemi miktarı · `b` = ürünün güvenlik stoğu

YEŞİL

`s ≥ q + b`

Sipariş sonrası bile güvenlik stoğu korunuyor. Ürün verilir, stok düşer.

SARI

`q ≤ s < q + b`

Sipariş karşılanır fakat kritik seviyeye inilir. Uyarı + tedarik talebi.

KIRMIZI

`s < q`

Karşılanamaz. Kalem YETERSIZ, alarm, tedarik talebi; stok düşülmez.

```
// Domain/Kurallar/IsikKarari.cs — saf fonksiyon, yan etkisiz, kolay test edilir
public static LedRengi Belirle(int mevcut, int istenen, int guvenlik)
{
    if (istenen <= 0) throw new ArgumentOutOfRangeException(nameof(istenen));
    if (mevcut <  istenen)             return LedRengi.Kirmizi;
    if (mevcut <  istenen + guvenlik)  return LedRengi.Sari;
    return LedRengi.Yesil;
}
```

6.2 Atomik stok düşme — yarış koşulunun çözümü

Problem

İki operatör aynı anda aynı kutudan toplama yaparsa: ikisi de stoğu *okur* (10 adet), ikisi de yeterli görür (q=6), ikisi de düşer → stok −2 olur veya biri eli boş döner. Klasik **read-then-write** yarış koşulu.

Çözüm: Kontrol ve düşme ayrı iki adım değil, **tek bir koşullu UPDATE**'tir. Veritabanı satır kilidini kendisi koyar.

```
-- Etkilenen satır sayısı 1 ise düşme başarılı, 0 ise stok yetersiz.
UPDATE stok
   SET adet = adet - @q,
       versiyon = versiyon + 1,
       guncelleme = now()
 WHERE kutu_id = @kutuId
   AND adet >= @q
RETURNING adet AS sonraki_adet, adet + @q AS onceki_adet;
```

```
// Infrastructure/Persistence/StokDeposu.cs
public async Task<StokDusSonucu> DusAsync(long kutuId, int q, CancellationToken ct)
{
    await using var tx = await _db.Database.BeginTransactionAsync(ct);

    var satir = await _db.Database
        .SqlQuery<StokDusSatiri>($"""...""")   // yukarıdaki UPDATE ... RETURNING
        .FirstOrDefaultAsync(ct);

    if (satir is null)                      // 0 satır → stok yetersiz
        return StokDusSonucu.Yetersiz();

    _db.StokHareketleri.Add(new StokHareketi {   // aynı transaction içinde log
        KutuId = kutuId, Tip = HareketTipi.Cikis, Miktar = q,
        OncekiAdet = satir.OncekiAdet, SonrakiAdet = satir.SonrakiAdet,
        SiparisKalemiId = kalemId, PersonelId = personelId
    });
    await _db.SaveChangesAsync(ct);
    await tx.CommitAsync(ct);
    return StokDusSonucu.Basarili(satir.SonrakiAdet);
}
```

Üç savunma hattı birlikte çalışır: (1) `WHERE adet >= @q` koşulu, (2) tablodaki `CHECK (adet >= 0)` kısıtı, (3) `versiyon` alanıyla optimistic lock. Stok hareketi logu aynı transaction'da yazılır; "düştü ama loglanmadı" durumu oluşamaz.

6.3 Ne zaman düşeceğiz? — Rezervasyon modeli kararı

Seçenek	Nasıl çalışır	Değerlendirme
A) Sipariş kabulünde düş SEÇİLDİ	Sipariş başlatılırken stok atomik düşülür; LED ancak düşme başarılıysa yakılır	Yeşil ışık **garantidir** — ışık yandıysa ürün o siparişe ayrılmıştır. Kırmızı daha ışık yanmadan yakalanır. İptal durumunda iade (GIRIS) gerekir.
B) Fiziksel teyitte düş	Işık yanar, operatör alır, teyit gelince düşülür	Fiziksel gerçeğe daha yakın; ancak iki operatör aynı anda yeşil ışık görebilir ve biri eli boş döner. Reddedildi.

Karar (Hafta-1 sunumuyla uyumlu): "Önce stoğu düş, sonra siparişi onayla"

1. Sipariş başlatılır → her kalem için koşullu UPDATE denenir.
2. Başarılı → kalem `ISIK_YANDI`, LED yeşil/sarı yanar, `CIKIS` hareketi loglanır.
3. Başarısız (s<q) → kalem `YETERSIZ`, LED **hiç yanmaz**, alarm + tedarik talebi; sipariş `KILITLI` veya kısmi devam.
4. Operatör teyidi geldiğinde kalem `ALINDI` olur — bu adım stoğu **tekrar düşürmez**, yalnızca fiziksel alımı doğrular.
5. Sipariş iptal edilir veya teyit zaman aşımına uğrarsa rezerve miktar `GIRIS` hareketiyle iade edilir.

6.4 Kutu seçim algoritması (bir ürün birden çok kutudaysa)

1. Ürünün bağlı olduğu **ONLINE** kutular listelenir.
2. Stoğu `q`'yu tek başına karşılayan kutular arasından **en az stoğu olan** seçilir (FIFO'ya yakın davranır, kutuyu boşaltarak ikmali kolaylaştırır).
3. Hiçbir kutu tek başına yetmiyorsa: toplam stok yeterliyse kalem **bölünür** (iki kutudan toplama, iki LED sırayla yanar); yetmiyorsa KIRMIZI.
4. Hiç ONLINE kutu yoksa kalem `BEKLIYOR` kalır, `CIHAZ_OFFLINE` alarmı üretilir ve arayüz manuel toplama önerir.

v1 sadeleştirmesi: kalem bölme (3. madde) opsiyoneldir, 6. haftada zaman kalırsa eklenir. Varsayılan davranış tek kutudan toplamadır.

7. Sipariş Yaşam Döngüsü ve Durum Makineleri

Sipariş girişinden ışığın sönmesine kadar tüm akış ve geçerli durum geçişleri.

7.1 Uçtan uca akış

```
[1] SİPARİŞ GİRİŞİ        Operatör arayüzden sipariş no + kalemleri girer
       |                  → siparis (BEKLIYOR) + siparis_kalemi (BEKLIYOR)
       ▼
[2] EŞLEŞTİRME            Her kalem için uygun kutu seçilir (Bölüm 6.4)
       |                  → siparis_kalemi.kutu_id doldurulur
       ▼
[3] STOK KONTROL + DÜŞME  Koşullu atomik UPDATE (Bölüm 6.2)
       |
       ├── başarılı ──► renk = Belirle(s,q,b) → YESIL | SARI
       |                kalem: ISIK_YANDI · stok_hareketi: CIKIS
       |                     |
       |                     ▼
       |               [4] LED YAK — MQTT: depo/kutu/{id}/komut  (QoS 1)
       |                     |      ESP32 ışığı yakar, miktarı gösterir
       |                     ▼
       |               [5] ALIM & TEYİT — buton / IR / ağırlık
       |                     |      MQTT: depo/kutu/{id}/olay
       |                     ▼
       |               [6] ONAY — kalem: ALINDI, alinma_zamani yazılır
       |                     |      LED söner, SignalR ile arayüz güncellenir
       |                     |      SARI ise → tedarik_talebi (ACIK)
       |                     ▼
       |                tüm kalemler ALINDI → siparis: TAMAMLANDI
       |
       └── başarısız (s < q) ──► renk = KIRMIZI, LED YAKILMAZ
                                 kalem: YETERSIZ · alarm: STOK_YOK
                                 ekranda kırmızı uyarı · tedarik_talebi
                                 sipariş: KILITLI (veya kısmi devam)
```

7.2 Sipariş durum makinesi

Durum	Anlamı	İzin verilen geçişler
`BEKLIYOR`	Girildi, henüz başlatılmadı	→ HAZIRLANIYOR, → KILITLI (başlatıldı ama hiçbir kalem karşılanamadı), → IPTAL
`HAZIRLANIYOR`	En az bir kalemin ışığı yandı	→ TAMAMLANDI, → KISMI_TAMAMLANDI, → KILITLI, → IPTAL
`KILITLI`	Bir kalem karşılanamadı, müdahale bekliyor	→ HAZIRLANIYOR (stok ikmal edildi, yeniden dene), → KISMI_TAMAMLANDI, → IPTAL
`KISMI_TAMAMLANDI`	Bazı kalemler alındı, bazıları yetersiz	→ TAMAMLANDI (eksikler sonradan tamamlandı), → IPTAL
`TAMAMLANDI`	Tüm kalemler ALINDI	son durum (değişmez)
`IPTAL`	İptal edildi; rezerve stok iade edildi	son durum

7.3 Sipariş kalemi durum makinesi

Durum	Anlamı	Çıkış geçişleri
`BEKLIYOR`	Kutu eşleşmedi veya kutu offline	→ ISIK_YANDI, → YETERSIZ, → IPTAL
`ISIK_YANDI`	Stok rezerve edildi, LED yanıyor	→ ALINDI (teyit), → IPTAL (zaman aşımı/iptal → stok iade)
`ALINDI`	Fiziksel alım teyit edildi	son durum
`YETERSIZ`	s < q; ışık yakılmadı	→ ISIK_YANDI (ikmal sonrası yeniden deneme), → IPTAL
`IPTAL`	İptal; rezerve varsa iade edildi	son durum

> **Zaman aşımı kuralı**
>
> `ISIK_YANDI` durumundaki bir kalem **5 dakika** (yapılandırılabilir) içinde teyit almazsa: LED söner, `TEYIT_ZAMAN_ASIMI` alarmı üretilir, rezerve stok `GIRIS` hareketiyle iade edilir ve kalem `BEKLIYOR`'a döner. Bu işi bir arka plan servisi (`ZamanAsimiKontrolServisi : BackgroundService`) 30 saniyede bir yapar.

8. Backend Tasarımı ve REST API Sözleşmesi

Servis sorumlulukları, uç noktalar ve canlı olay akışı.

8.1 Servisler

Servis	Sorumluluğu
`SiparisServisi`	Sipariş oluşturma, başlatma, iptal, durum geçişlerinin doğrulanması
`StokServisi`	Atomik düşme, iade, sayım, hareket loglama
`IsikKarariServisi`	Kutu seçimi + renk hesabı (Domain kuralını kullanır)
`CihazServisi`	Kutu kayıt/eşleme, LED komutu yayınlama, bağlantı durumu takibi
`AlarmServisi`	Alarm üretme, okundu işaretleme, tedarik talebi açma (mükerrer engelli)
`BildirimServisi`	SignalR üzerinden arayüze canlı olay gönderme
`MqttHostedService`	Broker bağlantısı, abonelik, gelen olayların iş servislerine yönlendirilmesi, yeniden bağlanma
`ZamanAsimiKontrolServisi`	Periyodik olarak süresi dolan rezervasyonları iade eder

8.2 REST uç noktaları

Metot	Yol	Açıklama	Rol
POST	`/api/auth/giris`	JWT access + refresh token üretir	—
POST	`/api/siparisler`	Sipariş + kalemlerini oluşturur (BEKLIYOR)	OP
GET	`/api/siparisler?durum=&sayfa=`	Sipariş listesi (sayfalı)	OP
GET	`/api/siparisler/{siparisNo}`	Sipariş detayı + kalemler + LED renkleri	OP
POST	`/api/siparisler/{id}/baslat`	**Ana akış:** eşleştir → stok düş → LED yak	OP
POST	`/api/siparisler/{id}/iptal`	Siparişi iptal eder, rezerve stoğu iade eder	OP
POST	`/api/kalemler/{id}/teyit`	Manuel teyit (sensör çalışmazsa yedek yol)	OP
GET	`/api/stok`	Tüm kutuların stok durumu + renk önizlemesi	OP
POST	`/api/stok/{kutuId}/sayim`	Fiziksel sayım düzeltmesi (SAYIM hareketi)	ADMIN
POST	`/api/stok/{kutuId}/giris`	Mal kabul / ikmal (GIRIS hareketi)	ADMIN
GET	`/api/urunler` · POST · PUT	Ürün kataloğu, güvenlik stoğu ayarı	ADMIN
GET	`/api/kutular`	Kutu listesi + bağlantı durumu + son görülme	OP
POST	`/api/kutular/{id}/led-test`	Bakım: seçilen renkte test yakma	ADMIN
GET	`/api/alarmlar?okundu=false`	Alarm listesi	OP
POST	`/api/alarmlar/{id}/okundu`	Alarmı okundu işaretle	OP
GET	`/api/tedarik-talepleri`	Açık ikmal talepleri	ADMIN
GET	`/api/hareketler?kutuId=&tarih=`	Stok hareket defteri (rapor)	ADMIN

Örnek: sipariş başlatma yanıtı

```
POST /api/siparisler/42/baslat  →  200 OK
{
  "siparisNo": "SIP-2026-00042",
  "durum": "HAZIRLANIYOR",
  "kalemler": [
    { "kalemId": 101, "urun": "Kablo 2m", "kutuKod": "K-01", "rafKonumu": "A1-R2",
      "miktar": 3, "kalanStok": 14, "ledRengi": "YESIL", "durum": "ISIK_YANDI" },
    { "kalemId": 102, "urun": "Adaptör 5V", "kutuKod": "K-04", "rafKonumu": "A2-R1",
      "miktar": 2, "kalanStok": 3,  "ledRengi": "SARI",  "durum": "ISIK_YANDI",
      "uyari": "Stok güvenlik seviyesinin altına indi — tedarik talebi açıldı" },
    { "kalemId": 103, "urun": "Sensör IR",  "kutuKod": "K-07", "rafKonumu": "B1-R3",
      "miktar": 5, "kalanStok": 1,  "ledRengi": "KIRMIZI", "durum": "YETERSIZ",
      "hata": "STOK_YETERSIZ: istenen 5, mevcut 1" }
  ],
  "alarmlar": [ { "tip": "STOK_YOK", "mesaj": "K-07 / Sensör IR: 4 adet eksik" } ]
}
```

8.3 SignalR canlı olayları (`/hubs/depo`)

Olay	Payload	Arayüzdeki etkisi
`StokGuncellendi`	kutuId, yeniAdet, renk	Kutu kartı ve stok tablosu anında güncellenir
`KutuIsikDegisti`	kutuId, renk, miktar, kalemId	Pano üzerinde kutu yanıp söner
`KalemTeyitEdildi`	kalemId, yontem, zaman	Toplama listesinde satır yeşile döner
`SiparisDurumDegisti`	siparisId, durum	Sipariş başlığı ve listeler güncellenir
`AlarmOlustu`	tip, mesaj, kutuId	Kırmızı toast + alarm rozetinde sayaç artışı
`CihazDurumDegisti`	kutuId, baglanti	Kutu kartı gri (OFFLINE) olur

8.4 Hata sözleşmesi

Tüm hatalar **RFC 7807 Problem Details** formatında döner; arayüz tek bir yerde yorumlar.

```
{ "type": "https://pick-to-light/errors/stok-yetersiz",
  "title": "Stok yetersiz", "status": 409,
  "detail": "K-07 kutusunda 1 adet var, 5 adet istendi.",
  "kutuKod": "K-07", "mevcut": 1, "istenen": 5 }
```

Kod	Durum
400	Doğrulama hatası (miktar ≤ 0, boş sipariş no)
401 / 403	Kimlik doğrulanmadı / rol yetersiz
404	Sipariş, ürün veya kutu bulunamadı
409	**Stok yetersiz**, mükerrer sipariş no, geçersiz durum geçişi
422	Ürün için tanımlı kutu yok / tüm kutular OFFLINE
503	MQTT broker erişilemiyor (komut kuyruğa alındı bilgisiyle)

9. MQTT Haberleşme Sözleşmesi

Yazılım ve elektrik ekiplerinin ortak dili. 2. hafta sonunda dondurulur, sonra değiştirilmez.

9.1 Topic şeması

Topic	Yön	QoS	İçerik
`depo/kutu/{kutuId}/komut`	Sunucu → Cihaz	1	LED yak / söndür / test komutu
`depo/kutu/{kutuId}/olay`	Cihaz → Sunucu	1	Teyit (buton/IR/ağırlık), buton basımı, hata
`depo/kutu/{kutuId}/durum`	Cihaz → Sunucu	1 · **retained**	ONLINE/OFFLINE, firmware sürümü, RSSI
`depo/kutu/{kutuId}/telemetri`	Cihaz → Sunucu	0	Ağırlık ölçümü, sıcaklık (opsiyonel, 10 sn periyot)
`depo/sistem/duyuru`	Sunucu → Tümü	1	Toplu söndürme, bakım modu

Sunucu `depo/kutu/+/olay` ve `depo/kutu/+/durum` joker aboneliklerini kullanır; yeni kutu eklendiğinde kod değişmez (NFR-06).

9.2 Mesaj formatları

LED yakma komutu (sunucu → cihaz)

```
{ "mesajId": "8f1c...-uuid",     // idempotenlik + teyit eşleştirme anahtarı
  "tip": "LED_YAK",
  "renk": "YESIL",               // YESIL | SARI | KIRMIZI
  "miktar": 3,                    // kaç adet alınacak (7-segment/yanıp sönme ile gösterilebilir)
  "kalemId": 101,
  "ttlSn": 300,                   // bu süre sonunda cihaz ışığı kendi söndürür
  "zaman": "2026-03-10T09:14:22Z" }
```

Teyit olayı (cihaz → sunucu)

```
{ "mesajId": "8f1c...-uuid",     // komuttan gelen id aynen geri döner
  "tip": "TEYIT",                 // TEYIT | IPTAL | HATA | BUTON
  "yontem": "BUTON",              // BUTON | IR | AGIRLIK
  "olculenFark": -148.5,          // gram (yalnız AGIRLIK yönteminde)
  "cihazMac": "A0:B1:C2:D3:E4:F5",
  "zaman": "2026-03-10T09:14:31Z" }
```

Durum / LWT mesajı (retained)

```
// Cihaz bağlanınca yayınlar:
{ "baglanti": "ONLINE", "fw": "1.0.2", "rssi": -58, "ip": "192.168.1.41" }

// Last Will & Testament — broker, cihaz koptuğunda bunu otomatik yayınlar:
{ "baglanti": "OFFLINE" }
```

9.3 Güvenilirlik kuralları

Kural	Uygulama
İdempotenlik	Her komut bir `mesajId` taşır; teyit aynı id ile döner. Sunucu bu id'yi `siparis_kalemi.mesaj_id` UNIQUE alanına yazar — tekrar gelen teyit sessizce yok sayılır (NFR-03).
QoS 1 (en az bir kez)	Komut ve teyitler kaybolmaz; tekrar gelme ihtimali idempotenlikle karşılanır.
Retained durum	Sunucu yeniden başladığında tüm kutuların güncel ONLINE/OFFLINE durumunu anında öğrenir.
LWT	Kutu elektriği giderse broker 30 sn içinde OFFLINE yayınlar; kutu panoda grileşir.
Keep-alive	20 sn; `kutu.son_gorulme` her mesajda güncellenir.
Yeniden bağlanma	Hem cihaz hem sunucu tarafında üstel geri çekilme (1→2→4→8→30 sn tavan).
Ham log	Gelen her mesaj `cihaz_olayi` tablosuna JSONB olarak yazılır — hata ayıklamanın tek kaynağı.
Cihaz kimliği	Her ESP32 kendi MQTT kullanıcı adı/parolasıyla bağlanır; broker ACL'i ile yalnızca kendi topic'ine yazabilir.

> **Sanal kutu simülatörü (NFR-10)**
>
> Donanım hazır olmadan yazılımın ilerleyebilmesi için küçük bir .NET konsol uygulaması (`tools/KutuSimulatoru`) yazılır: aynı topic'lere abone olur, LED komutu alınca ekrana renk basar ve 2 sn sonra teyit mesajı döner. Yazılım ekibi 3. haftadan itibaren tüm akışı bununla test eder.

10. Donanım Planı ve ESP32 Firmware Tasarımı

Elektrik ekibinin iş paketi: kare modüler kutu, LED sürücü, onay sensörü, güç ve firmware.

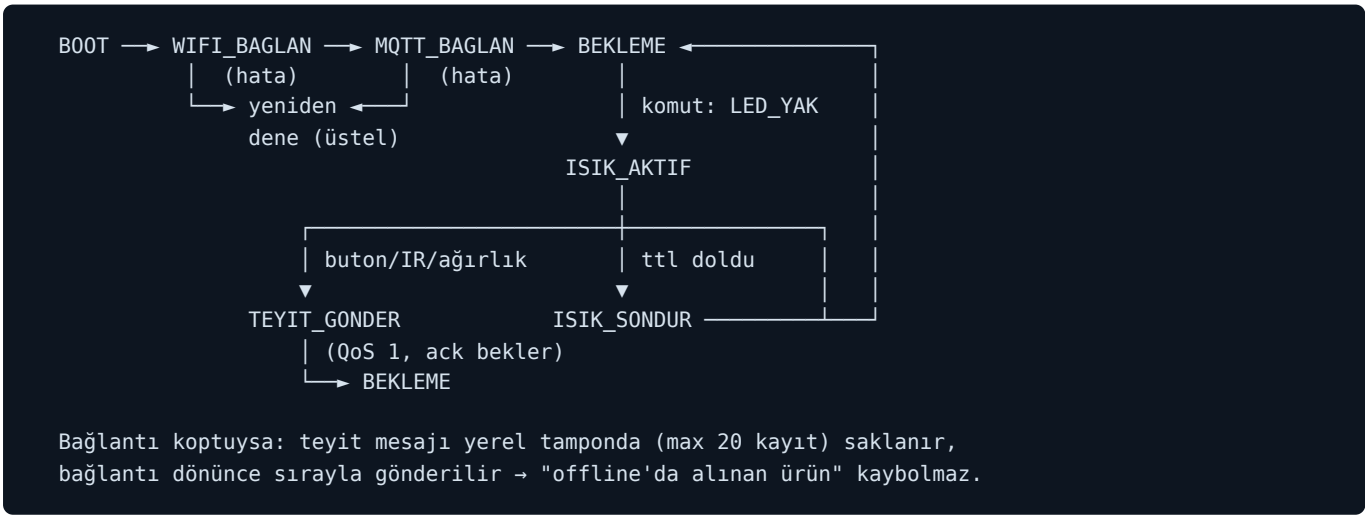
10.1 Kutu başına malzeme listesi (BOM)

Bileşen	Seçim	Adet	Not
Mikrodenetleyici	ESP32-WROOM-32 DevKit	1	Dahili Wi-Fi; LED sürer, sensör okur, MQTT konuşur
Durum LED'i	WS2812B (adreslenebilir RGB) *veya* 3'lü ayrı LED	1 / 3	WS2812B tek pinle üç rengi verir, kablolama azalır — önerilen
Direnç	220 Ω (ayrı LED kullanılırsa) · 10 kΩ pull-up	3 / 1	Akım sınırlama ve buton pull-up
Onay sensörü	Buton (v1) · TCRT5000 IR · HX711 + load cell	1	v1'de buton; IR/ağırlık 5-6. haftada eklenir
Güç	5 V 10 A ortak hat + kutu başına paralel besleme	—	Kutu başına ~100 mA (LED dahil); 8 kutu ≈ 1 A
Kasa	Kare modüler kutu (MDF/pleksi veya 3D baskı)	1	Yan yana çoğaltılabilir; ürün gözü + LED yuvası
Kablolama	JST konnektör, 2×0.5 mm² güç hattı	—	Ortak GND şart; konnektörle kutu sökülebilir olmalı

10.2 Pin planı (öneri)

Pin	Bağlantı	Açıklama
GPIO 5	WS2812B DIN	Veri hattı (330 Ω seri direnç önerilir)
GPIO 18 / 19 / 21	Yeşil / Sarı / Kırmızı LED	Ayrı LED kullanılırsa; her biri 220 Ω üzerinden
GPIO 4	Onay butonu	INPUT_PULLUP, 50 ms yazılımsal debounce
GPIO 32 / 33	HX711 DT / SCK	Ağırlık sensörü (opsiyonel)
GPIO 34	TCRT5000 OUT	IR mesafe sensörü (opsiyonel, sadece giriş)
5V / GND	Ortak güç hattı	Tüm kutular paralel; GND ortak olmalı

10.3 Firmware durum makinesi



10.4 Firmware gereksinimleri

- **Kimlik:** cihaz MAC adresi topic ve kimlik doğrulamada kullanılır; `kutuId` ilk kurulumda NVS'e yazılır.
- **Yanıt süresi:** komut alındıktan sonra LED < 200 ms içinde yanar.
- **Miktar gösterimi:** alınacak adet, LED'in o sayıda kısa yanıp sönmesiyle gösterilir (ek donanım gerektirmeyen çözüm).
- **Debounce:** buton için 50 ms; çift tetiklenme firmware'de engellenir, sunucu tarafında ayrıca `mesajId` ile korunur.
- **Ağırlık doğrulaması:** HX711 kullanılıyorsa alınan ağırlık `miktar × urun.birim_agirlik_g` ile ±%10 toleransla karşılaştırılır; uyuşmazsa `HATA` olayı gönderilir.
- **Güvenli varsayılan:** MQTT bağlantısı yoksa LED yakılmaz (yanlış yönlendirme riskini ortadan kaldırır); kutu üzerinde küçük bir "bağlantı yok" göstergesi (LED kısa mavi/beyaz nabız) kullanılır.
- **OTA (opsiyonel):** ArduinoOTA ile kablosuz firmware güncelleme — 7. haftada zaman kalırsa.

10.5 Elektrik ekibi teslim takvimi

Hafta	Teslim
H2	Kutu çizimi/ölçüleri, BOM onayı, 1 adet prototip kutu montajı
H3	ESP32 + LED + buton breadboard devresi; MQTT ile "merhaba" mesajı
H4	Sözleşmeye uygun firmware v1: komutla LED yakma + buton teyidi
H5	Kutu 2-3-4 çoğaltma, güç dağıtım hattı, kablolama düzeni
H6	Onay sensörü yükseltmesi (IR veya HX711), ağırlık kalibrasyonu
H7	Mekanik son montaj, etiketleme, dayanıklılık testi (100 toplama döngüsü)
H8	Demo düzeneği kurulumu ve yedek kutu/kablo hazırlığı

11. Frontend Tasarımı (Next.js) ve Ekran Planı

Operatör ekrana değil ışığa bakar; arayüz denetim ve istisna yönetimi içindir.

11.1 Ekranlar

Ekran	İçerik	Öncelik
Canlı Pano `/`	Kutu ızgarası (her kutu bir kart: kod, ürün, stok, renk, ONLINE/OFFLINE), aktif sipariş özeti, son alarmlar	Yüksek — demo ekranı
Sipariş Girişi `/siparis/yeni`	Sipariş no + kalem ekleme (ürün ara, miktar), "Siparişi Başlat" butonu	Yüksek
Toplama Ekranı `/siparis/{no}`	Kalem listesi: kutu kodu, raf konumu, miktar, renk rozeti, durum; manuel teyit butonu; ilerleme çubuğu	Yüksek
Alarm Paneli `/alarmlar`	Kırmızı/sarı uyarılar, okundu işaretleme, ilgili siparişe gitme	Yüksek
Stok Yönetimi `/stok`	Kutu-ürün-adet tablosu, mal kabul (GIRIS), sayım (SAYIM), güvenlik stoğu düzenleme	Orta
Kutu/Cihaz Yönetimi `/kutular`	Kutu ekleme, MAC-topic eşleme, LED test, son görülme zamanı	Orta
Raporlar `/raporlar`	Hareket defteri, en çok toplanan ürünler, ortalama toplama süresi	Düşük — zaman kalırsa
Giriş `/giris`	Personel kodu + parola, JWT alma	Yüksek

11.2 Kutu kartı bileşeni — görsel dil

Durum	Görünüm	Ek işaret (renk körlüğü için — NFR-08)
YEŞİL	Yeşil çerçeve + dolu nokta	✓ ikonu + "Yeterli" metni
SARI	Sarı çerçeve, hafif nabız animasyonu	⚠ ikonu + "Azaldı" metni
KIRMIZI	Kırmızı çerçeve, yanıp sönme + sesli uyarı (opsiyonel)	✕ ikonu + "Yetersiz" metni
OFFLINE	Gri, soluk kart	Fiş ikonu + "Bağlantı yok"

11.3 Veri akışı kalıbı

```
// Sunucu durumu TanStack Query'de; SignalR olayı geldiğinde ilgili sorgu tazelenir.
const { data: kutular } = useQuery({ queryKey: ['kutular'], queryFn: kutuApi.listele });

useSignalR('StokGuncellendi', (olay) => {
  qc.setQueryData(['kutular'], (eski) => guncelle(eski, olay));   // anlık yansıt
  qc.invalidateQueries({ queryKey: ['stok'] });                    // arka planda doğrula
});

useSignalR('AlarmOlustu', (a) => toast.hata(a.mesaj));
```

11.4 Klasör yapısı

```
web/
├─ src/app/                 // App Router — her klasör bir rota
│   ├─ layout.tsx · page.tsx            // kök düzen + canlı pano
│   ├─ siparis/yeni/page.tsx · siparis/[no]/page.tsx
│   ├─ stok/page.tsx · kutular/page.tsx · alarmlar/page.tsx
│   └─ giris/page.tsx
├─ src/lib/      (apiClient.ts, signalr.ts, auth.ts)
├─ src/hooks/    (useSignalR, useSiparis, useKutular)
├─ src/components/ (KutuKarti, RenkRozeti, KalemSatiri, AlarmToast)
├─ src/types/    (backend DTO'larıyla birebir eşleşen arayüzler)
└─ next.config.ts · tailwind.config.ts
```

12. Güvenlik, Yetkilendirme ve İzlenebilirlik

12.1 Kimlik doğrulama ve yetki

Konu	Yaklaşım
Kimlik doğrulama	JWT Bearer — access token 15 dk, refresh token 7 gün (HttpOnly cookie)
Parola saklama	ASP.NET Core Identity `PasswordHasher` (PBKDF2); düz parola asla saklanmaz
Roller	`ADMIN` : ürün/kutu/eşik yönetimi, sayım, raporlar · `OPERATOR` : sipariş toplama, teyit, alarm okuma
Yetki uygulaması	Controller seviyesinde `[Authorize(Roles = "ADMIN")]` ; kritik işlemler ayrıca servis katmanında kontrol edilir
Cihaz kimliği	Her ESP32 için ayrı MQTT kullanıcı/parola + broker ACL: cihaz yalnızca kendi `depo/kutu/{id}/*` topic'ine yazabilir
Taşıma güvenliği	Web: HTTPS · MQTT: v1'de yerel ağda 1883; demo sonrası TLS (8883) opsiyonu
Sırlar	Geliştirmede `dotnet user-secrets` , çalıştırmada ortam değişkeni; bağlantı dizeleri depoya **asla** commit edilmez
Hız sınırlama	ASP.NET Core Rate Limiting: giriş uç noktası dakikada 5 deneme
CORS	Yalnızca arayüz origin'ine izin

12.2 İzlenebilirlik (audit)

- **Stok hareket defteri** (`stok_hareketi`): her değişimde önceki adet, sonraki adet, tip, personel, sipariş kalemi, zaman — **append-only**, güncelleme/silme yoktur.
- **Cihaz olay logu** (`cihaz_olayi`): gelen ham MQTT mesajı JSONB olarak; "ışık neden yanmadı" sorusunun cevabı burada.
- **Uygulama logu** (Serilog): yapılandırılmış JSON log; her isteğe `CorrelationId` eklenir, MQTT mesajına da taşınır — tek bir toplama olayı uçtan uca izlenebilir.
- **Sağlık kontrolü**: `/health` uç noktası DB + MQTT broker bağlantısını raporlar.

13. Hata Senaryoları ve Kurtarma Stratejisi

"Her şey yolunda gittiğinde" çalışan sistem kolaydır; not bu tablodan alınır.

Senaryo	Sistem davranışı	Kurtarma
Kutu OFFLINE (elektrik/Wi-Fi gitti)	LWT ile kutu gri; o kutuya komut gönderilmez, kalem `BEKLIYOR` kalır	`CIHAZ_OFFLINE` alarmı; arayüz "manuel topla + manuel teyit ver" yolunu açar
MQTT broker erişilemiyor	Komutlar bellek içi kuyruğa alınır; API 503 + "komut kuyrukta" bilgisi döner	Yeniden bağlanınca kuyruk boşaltılır; 60 sn'yi aşarsa rezervasyonlar iade edilir
Teyit mesajı hiç gelmiyor	Kalem `ISIK_YANDI` 'da takılı kalır	5 dk sonra zaman aşımı: LED söner, stok iade edilir, `TEYIT_ZAMAN_ASIMI` alarmı (Bölüm 7.3)
Aynı teyit iki kez geliyor (QoS 1 tekrarı)	İkinci mesaj `mesajId` çakışmasından yakalanır	Sessizce yok sayılır, stok ikinci kez düşmez (NFR-03)
İki sipariş aynı kutudan eşzamanlı istiyor	Koşullu UPDATE birini başarısız kılar	İkincisine KIRMIZI/alternatif kutu önerilir; negatif stok oluşmaz (FR-06)
Operatör yanlış kutudan ürün alıyor	Beklenmeyen kutudan teyit gelir; eşleşen aktif kalem yoktur	Olay `cihaz_olayi` 'na yazılır, `BEKLENMEYEN_TEYIT` alarmı üretilir, stok **düşülmez**
Ağırlık sensörü uyumsuzluk bildiriyor	Alınan ağırlık beklenenin ±%10 dışında	Teyit kabul edilir ama "sayım farkı" uyarısı; sorumluya sayım önerilir
Fiziksel stok ile kayıt uyuşmuyor	Sayım ekranından fark girilir	`SAYIM` hareketi, fark loglanır, sapma raporuna düşer
Veritabanı kısıt ihlali (adet < 0)	Transaction geri alınır	Hiçbir kısmi değişiklik kalmaz; hata loglanır ve 409 döner
Sunucu yeniden başlıyor	Retained durum mesajlarıyla kutu durumları geri yüklenir	Açık rezervasyonlar DB'den okunur; süresi geçenler iade edilir
Sipariş iptal edildi	Rezerve kalemler bulunur	Her biri için `GIRIS` hareketi, LED söndürme komutu, kalem `IPTAL` (FR-13)

14. Test Planı ve Kabul Kriterleri

14.1 Test piramidi

Seviye	Araç	Kapsam
Birim	xUnit + FluentAssertions	Işık kararı (sınır değerler), durum makinesi geçişleri, kutu seçim algoritması
Entegrasyon	Testcontainers (PostgreSQL + Mosquitto)	Atomik stok düşme, hareket loglama, MQTT komut/olay turu, idempotenlik
Eşzamanlılık	xUnit + `Parallel.ForEachAsync`	100 paralel toplama isteği → negatif stok yok, toplam düşüm = başlangıç stoğu
API sözleşme	WebApplicationFactory	Uç nokta yanıt kodları, Problem Details formatı, yetki kontrolleri
E2E (arayüz)	Playwright	Sipariş gir → başlat → simülatörden teyit → stok düştü ekranda görüldü
Donanım-in-the-loop	Gerçek ESP32 + manuel senaryo	LED yanma süresi, buton debounce, Wi-Fi kopma/dönme, 100 döngü dayanıklılık

14.2 Işık kararı birim test tablosu (sınır değer analizi)

s (mevcut)	q (istenen)	b (güvenlik)	Beklenen	Neden
20	3	5	YEŞİL	20 ≥ 8
8	3	5	YEŞİL	tam sınır: s = q+b
7	3	5	SARI	3 ≤ 7 < 8
3	3	5	SARI	tam sınır: s = q
2	3	5	KIRMIZI	2 < 3
0	1	5	KIRMIZI	stok tükendi
5	5	0	YEŞİL	güvenlik stoğu 0 ise s=q yeşildir

14.3 Kabul kriterleri (UAT — 8. hafta)

No	Senaryo	Beklenen sonuç
UAT-1	3 kalemli sipariş girilir ve başlatılır	3 kutunun LED'i doğru renkte < 1 sn içinde yanar
UAT-2	Yeşil yanan kutudan ürün alınır, butona basılır	LED söner, stok ekranda anında azalır, hareket loglanır
UAT-3	Stoğu güvenlik seviyesine yakın üründen talep edilir	Sarı yanar, tedarik talebi açılır, uyarı gösterilir
UAT-4	Stoktan fazla miktar talep edilir	LED yanmaz, ekranda kırmızı alarm, sipariş kilitlenir, stok değişmez
UAT-5	İki sipariş aynı kutudan eşzamanlı başlatılır	Yalnızca stoğun yettiği kadarı rezerve edilir; negatif stok yok
UAT-6	Bir kutunun fişi çekilir	30 sn içinde panoda OFFLINE; o kutuya komut gönderilmez, alarm üretilir
UAT-7	Işık yandıktan sonra teyit verilmez (5 dk beklenir)	LED söner, stok iade edilir, zaman aşımı alarmı düşer
UAT-8	Sipariş iptal edilir	Rezerve stok geri yüklenir, GIRIS hareketi loglanır, LED'ler söner
UAT-9	Sayım ile stok düzeltilir	SAYIM hareketi oluşur, fark raporda görünür
UAT-10	OPERATOR rolüyle güvenlik stoğu değiştirilmeye çalışılır	403 döner, işlem gerçekleşmez

15. 8 Haftalık Proje Planı ve İş Kırılım Yapısı

Hafta-1 sunumundaki yol haritasının göreve dökülmüş hâli. "Y" = Yazılım ekibi, "E" = Elektrik ekibi.

15.1 Hafta hafta plan

Hafta	Tema	Görevler	Hafta sonu teslimi
H1 şu an	Fikir, gereksinim, planlama	Y+E: problem analizi, sunum, gereksinim listesi, ER diyagramı, **bu planlama dokümanı**	Onaylanmış kapsam ve teknoloji yığını
H2	Mimari + iskelet	Y: solution yapısı (4 katman), EF Core şeması + ilk migration, seed verisi, Docker Compose (Postgres + Mosquitto), **MQTT sözleşmesinin dondurulması** E: kutu çizimi, BOM onayı, 1 prototip kutu montajı	Boş ama çalışan API + ayağa kalkan veritabanı + 1 fiziksel kutu
H3	Çekirdek iş mantığı	Y: ışık kararı (birim testleriyle), atomik stok düşme, hareket loglama, sipariş CRUD uçları, **kutu simülatörü** E: ESP32 + LED + buton devresi, MQTT'ye ilk bağlantı	Simülatör üzerinden uçtan uca akış (donanımsız)
H4	LED mantığı + MQTT entegrasyonu	Y: `MqttHostedService`, komut yayını, olay işleme, idempotenlik, alarm servisi E: firmware v1 — sözleşmeye uygun komut alma + buton teyidi	**Gerçek kutu gerçek komutla yanıyor** (kilometre taşı)
H5	Çoklu kutu + arayüz başlangıcı	Y: SignalR hub + canlı pano, sipariş giriş ekranı, giriş/yetkilendirme E: kutu 2-3-4 çoğaltma, güç dağıtımı, kablolama	4 kutu + canlı pano üzerinde izlenebilen akış
H6	Otomatik stok düşme + alarm ekranı	Y: teyit→stok düşme zinciri, zaman aşımı servisi, alarm/tedarik ekranları, stok yönetimi E: onay sensörü yükseltmesi (IR / HX711), kalibrasyon	Tam akış: ışık → alım → otomatik düşme → alarm
H7	Sağlamlaştırma + testler	Y: entegrasyon ve eşzamanlılık testleri, hata senaryoları, iptal/iade akışı, raporlar, performans ölçümü E: mekanik son montaj, 100 döngü dayanıklılık testi, etiketleme	Test raporu + kararlı sistem
H8	Uçtan uca test, demo, sunum	Y+E: UAT senaryolarının koşulması, demo verisi, kurulum dokümanı, final sunumu, yedek plan (plan B videosu)	**Çalışan prototip + final sunum**

15.2 Kilometre taşları

Tarih	Kilometre taşı	Neden kritik
H2 sonu	MQTT sözleşmesi dondurulduu	İki ekibin paralel çalışabilmesinin ön şartı; geciken her gün iki ekibi birden bloke eder
H3 sonu	Simülatörle uçtan uca akış	Donanım gecikse bile yazılım ilerleyebilir (risk R-1'in önlemi)
H4 sonu	İlk gerçek LED yanması	Projenin "çalışıyor" kanıtı; entegrasyon riskinin en erken ölçüldüğü an
H6 sonu	Özellik dondurma (feature freeze)	Son iki hafta yalnızca test, düzeltme ve sunuma ayrılır
H8	Demo + teslim	UAT senaryolarının tamamı geçmiş olmalı

15.3 Görev dağılımı özeti

Elektrik Ekibi

- Kare modüler kutu tasarımı ve montajı
- LED sürücü devresi ve dirençler
- ESP32 programlama (Arduino/PlatformIO)
- Onay sensörü (buton / IR / ağırlık) entegrasyonu
- 5 V güç dağıtımı ve kablolama
- Mekanik dayanıklılık testi

Yazılım Ekibi

- Backend API ve iş mantığı (.NET 9)
- Veritabanı şeması ve stok modeli (PostgreSQL)
- Sipariş & otomatik stok düşme akışı
- Web arayüzü ve canlı alarm ekranı (Next.js)
- MQTT haberleşme ve bildirim servisi
- Test altyapısı ve kutu simülatörü

Ortak sorumluluk: MQTT sözleşmesi (H2), entegrasyon testleri (H4, H7), demo kurulumu (H8). Haftada bir 30 dakikalık ortak senkron toplantı önerilir; sözleşme değişikliği iki ekibin onayı olmadan yapılmaz.

16. Risk Analizi

O = Olasılık, E = Etki (1 düşük – 3 yüksek).

No	Risk	O	E	Önlem / plan B
R-1	Donanım malzemesi geç gelir, ESP32/sensör tedarik edilemez	2	3	Kutu simülatörü (H3) ile yazılım bağımsız ilerler; demo gerekirse 2 kutuyla yapılır; yedek ESP32 ve LED önceden alınır
R-2	Wi-Fi kopmaları, MQTT kararsızlığı	3	2	QoS 1 + LWT + yeniden bağlanma + cihazda yerel tampon; demo için ayrı/temiz bir SSID kullanılır
R-3	Eşzamanlı toplamada stok tutarsızlığı	2	3	Koşullu atomik UPDATE + CHECK kısıtı + eşzamanlılık testi (Bölüm 6.2, 14.1)
R-4	İki ekibin entegrasyonu geç kalır, son hafta "birleştirme krizi"	2	3	Sözleşmenin H2'de dondurulması; H4'te ilk gerçek entegrasyon kilometre taşı; haftalık senkron
R-5	Ağırlık sensörü (HX711) kalibrasyonu güvenilmez çıkar	3	1	v1'de teyit butonu esas; ağırlık yalnızca doğrulama amaçlı ikincil sinyal olarak kullanılır
R-6	Kapsam şişmesi (barkod, rota optimizasyonu, mobil uygulama istekleri)	3	2	Bölüm 2.2'deki kapsam dışı listesi referans alınır; H6'da özellik dondurma
R-7	Teknoloji öğrenme eğrisi (.NET + MQTT + React aynı anda)	2	2	H2-H3'te küçük "dikey dilim" (tek kutu, tek ürün) üzerinden öğrenme; çift programlama
R-8	Güç dağıtımı sorunları (gerilim düşümü, ortak GND eksikliği)	2	2	Ortak GND kuralı, yeterli kesitli kablo, 10 A kaynak; kutu başına akım ölçümü H5'te yapılır
R-9	Demo günü donanım arızası	2	3	Yedek kutu ve kablo seti; önceden çekilmiş demo videosu; simülatör modu ile sunum yapılabilmesi
R-10	Ekip üyesi devamsızlığı / zaman çakışması	2	2	Görevlerin tek kişiye bağımlı olmaması, repoda dokümantasyon, haftalık ilerleme kaydı

17. Geliştirme Ortamı, Repo Yapısı ve Çalışma Düzeni

17.1 Depo yapısı

```
Pick-to-Light/
├─ docs/                  // bu doküman, ER diyagramı, sunumlar, kurulum notları
├─ src/
│  ├─ PickToLight.Domain/
│  ├─ PickToLight.Application/
│  ├─ PickToLight.Infrastructure/
│  └─ PickToLight.Api/
├─ web/                   // Next.js (App Router) arayüz
├─ firmware/
│  └─ kutu-esp32/         // PlatformIO projesi (elektrik ekibi)
├─ tools/
│  └─ KutuSimulatoru/     // donanımsız test için sanal kutu
├─ tests/
│  ├─ Domain.UnitTests/
│  ├─ Application.UnitTests/
│  └─ Api.IntegrationTests/
├─ docker-compose.yml     // postgres + mosquitto + api + web
└─ README.md
```

17.2 Yerel ortam

```
# Tüm altyapı tek komutla
docker compose up -d        # postgres:17 · eclipse-mosquitto:2 · api · web
dotnet ef database update   # şema + seed verisi
dotnet run --project src/PickToLight.Api
cd web && npm run dev        # http://localhost:3000
dotnet run --project tools/KutuSimulatoru -- --kutu 1,2,3,4
```

17.3 Çalışma düzeni

Konu	Kural
Dal (branch) stratejisi	`main` daima çalışır durumda; özellik dalları `feature/stok-dusme`, `feature/mqtt-servis` biçiminde
Commit mesajı	Türkçe veya İngilizce, tek biçim: `feat: atomik stok düşme eklendi`
Kod incelemesi	Her PR en az bir ekip arkadaşı tarafından okunur; iş mantığı değişen PR'larda test zorunlu
Tanımlar	Veritabanı ve API alan adları Türkçe (ER ile uyumlu), kod içi teknik terimler İngilizce karışımı kabul edilir — ER diyagramındaki isimlendirme esastır
Dokümantasyon	MQTT sözleşmesi ve API değişiklikleri `docs/` altında aynı PR'da güncellenir
Haftalık ritim	Pazartesi plan, Cuma demo+retrospektif (15 dk); ilerleme bu dokümanın 15.1 tablosuna işlenir

18. Demo Senaryosu ve Başarı Ölçütleri

18.1 8. hafta demo akışı (≈ 6 dakika)

1. **Kurulum tanıtımı (45 sn)** — 4 kutu, her birinin LED'i ve butonu, ekranda canlı pano; kutuların ONLINE olduğu gösterilir.
2. **Yeşil senaryo (90 sn)** — `SIP-001` siparişi girilir ve başlatılır; iki kutu yeşil yanar. Ürünler alınır, butonlara basılır; LED'ler söner, stok ekranda anında azalır, hareket defteri gösterilir.
3. **Sarı senaryo (60 sn)** — stoğu kritik seviyeye yakın üründen sipariş verilir; sarı yanar, uyarı ve otomatik açılan tedarik talebi gösterilir.
4. **Kırmızı senaryo (60 sn)** — stoktan fazla miktar istenir; **LED yanmaz**, ekranda kırmızı alarm belirir, siparişin kilitlendiği ve stoğun değişmediği gösterilir.
5. **Dayanıklılık gösterimi (60 sn)** — bir kutunun fişi çekilir, panoda OFFLINE'a düştüğü ve alarmın oluştuğu gösterilir; fiş takılınca otomatik geri döndüğü izlenir.
6. **Eşzamanlılık (45 sn)** — aynı kutudan iki sipariş aynı anda başlatılır; stoğun negatife düşmediği, ikinci siparişin uyarı aldığı gösterilir.
7. **Kapanış (30 sn)** — mimari şeması, 8 haftalık yol ve ekip katkıları.

18.2 Başarı ölçütleri

Ölçüt	Hedef	Ölçüm yöntemi
Sipariş başlatmadan LED yanmasına süre	< 1 sn	Log zaman damgaları (API isteği → cihaz olayı)
Eşzamanlı istekte stok tutarsızlığı	0 vaka	100 paralel istek testi + DB doğrulaması
Teyit edilen alımların loglanma oranı	%100	`stok_hareketi` ile `siparis_kalemi` karşılaştırması
Geçen UAT senaryosu	10/10	Bölüm 14.3 kontrol listesi
Eşzamanlı çalışan kutu	≥ 4	Demo düzeneği
Kutu kopma sonrası tespit süresi	< 30 sn	LWT + keep-alive testi

19. Sözlük

Terim	Anlamı
Pick-to-Light	Toplanacak ürünün bulunduğu rafı/kutuyu ışıkla gösteren endüstriyel toplama sistemi
s / q / b	s = kutudaki mevcut stok · q = sipariş kalemi miktarı · b = ürünün güvenlik stoğu
Güvenlik stoğu (buffer)	Ürün bazında belirlenen asgari tampon; sarı ışık eşiğini oluşturur
Atomik işlem	Ya tamamı uygulanan ya da hiç uygulanmayan, araya başka işlem giremeyen veritabanı işlemi
Yarış koşulu (race condition)	İki işlemin aynı veriyi eşzamanlı okuyup yazmasından doğan tutarsızlık
Rezervasyon	Sipariş kabulünde stoğun o siparişe ayrılması (bu projede doğrudan düşülerek yapılır)
İdempotenlik	Aynı isteğin birden çok kez işlenmesinin tek kez işlenmişle aynı sonucu vermesi
MQTT	Hafif yayınla/abone ol (publish/subscribe) mesajlaşma protokolü; IoT cihazlarının standardı
QoS 1	"En az bir kez teslim" garantisi; mesaj kaybolmaz ama tekrarlanabilir
LWT (Last Will & Testament)	Cihaz beklenmedik şekilde koptuğunda broker'ın onun adına yayınladığı mesaj
Retained mesaj	Broker'da saklanan son durum mesajı; yeni abone bağlanır bağlanmaz alır
SignalR	.NET'in gerçek zamanlı sunucu→tarayıcı iletişim kütüphanesi (WebSocket)
HIL (Hardware-in-the-loop)	Gerçek donanımın döngüye dahil edildiği test yöntemi
UAT	Kullanıcı kabul testi — sistemin işi gerçekten yapıp yapmadığının son kontrolü

Sonraki adım

Bu doküman onaylandıktan sonra 2. hafta işleri başlar: (1) solution iskeleti ve EF Core şeması, (2) Docker Compose ortamı, (3) MQTT sözleşmesinin dondurulması, (4) ilk prototip kutunun montajı. Kodlama aşamasına geçmeden önce Bölüm 5.3'teki ER eklemeleri ve Bölüm 6.3'teki rezervasyon modeli kararının ekipçe onaylanması önerilir.